ENGENHARIA DE SOFTWARE • RELATÓRIO DE REFLEXÃO CRÍTICA

Protótipo Gerado por IA vs. Engenharia de Software

Estudo de Caso: Dinamização de Visitantes no Museu Nacional Soares dos Reis (MNSR)

INSTITUIÇÃO	DURAÇÃO DO SPRINT	FERRAMENTAS IA	ENTREGÁVEL
MNSR (Porto)	**60–90 Minutos**	**Claude, ChatGPT, Cursor**	**Protótipo Móvel & AI Log**

1. CONTEXTO & DIAGNÓSTICO DO PROBLEMA DE NEGÓCIO

O Museu Nacional Soares dos Reis (MNSR), fundado em 1833 no Palácio dos Carrancas, é o museu público de arte mais antigo de Portugal. Apesar de albergar acervos ímpares de escultura (ex.: *O Desterro*) e pintura nacional, enfrenta um acentuado défice de atratividade em comparação com outros polos culturais do Porto. O desafio central consiste em alavancar tecnologia móvel para atrair público jovem, famílias e incentivar o regresso dos residentes locais.

85.000
Visitantes/Ano
< 8% do turismo cultural do Porto (Lello: 1,2M; Clérigos: 800k; Serralves: 500k).

15% / 12%
Jovens / Famílias
62% dos visitantes são idosos em excursões. Retorno de residentes < 5%/ano.

42 min
Tempo de Permanência
70% do tráfego concentrado entre as 11h e as 15h, gerando assimetrias nas salas.

2. PROTÓTIPO GERADO E REGISTO DE UTILIZAÇÃO DE IA (AI LOG)

Durante o sprint de 60 a 90 minutos, utilizaram-se assistentes de IA Generativa para produzir um protótipo funcional composto por: **Roteiros Inteligentes** (rotas de 30, 60 e 90 min), **Chatbot de Peças** (guia interativo para obras como *O Desterro*), **Caça ao Tesouro Gamificada** (QR Codes e descontos) e **Indicador de Afluência em Tempo Real**.

FUNCIONALIDADE	FERRAMENTA & PROMPTS SUBMETIDOS	RESULTADO GERADO PELA IA	AVALIAÇÃO AUDITADA
Roteiros Temáticos	*Claude / ChatGPT:* "Gera componente React com rotas dinâmicas de 30/60/90 min para o MNSR."	Interface limpa com seleção de tempos e cards de obras estáticos.	**Aceito c/ Reparos:** Dados em *hardcode*; sem API real.
Chatbot de Peças	*Cursor / Copilot:* "Cria widget de chat em JS que responda a dúvidas sobre 'O Desterro'."	Chat funcional com chamadas diretas à API de LLM com contexto embutido.	**Aceito Condicional:** Falta validação e limites de requisição.
Caça ao Tesouro (QR)	*Bolt.new:* "Implementa leitor de QR Code para dar badges e descontos na loja do museu."	Leitura funcional via câmara; estado guardado apenas em `localStorage`.	**Rejeitado (Backend):** Estado vulnerável a adulteração do utilizador.

3. ANÁLISE CRÍTICA DAS LIMITAÇÕES GRAVES DO PROTÓTIPO IA

Apesar de a IA Generativa ter produzido um protótipo visualmente convincente no "caminho feliz" (*happy path*), a auditoria técnica revelou falhas estruturais graves que impedem o seu lançamento em ambiente de produção:

1. Arquitetura e Modelação: Acoplamento Oculto e Inexistência de ADRs

ARQUITETURA

O código gerado funde a lógica de apresentação (UI), as regras de negócio (cálculo de pontos/badges) e o acesso a dados num único bloco monolítico. Não existem padrões de camada (Layered / Clean Architecture), resultando em elevado acoplamento. Adicionalmente, escasseiam decisões de arquitetura justificadas (ADRs - *Architecture Decision Records*), tornando impossível integrar o sistema com o inventário real do MNSR ou manter o código de forma sustentável por múltiplos programadores.

2. Garantia de Qualidade e Testes: Cobertura Nula e Vulnerabilidade a Exceções

TESTES & QA

O protótipo não contém quaisquer testes unitários, de integração ou de ponta a ponta (E2E). O sistema responde adequadamente apenas sob condições ideais. Falhas de ligação à rede nas salas do museu, inserção de dados inválidos pelo utilizador ou respostas desalinhadas (alucinações) do Chatbot resultam em bloqueios críticos da aplicação sem qualquer mecanismo de recuperação elegante ou *fallback*.

3. Segurança e Sustentabilidade: Chaves Expostas e Risco de Dívida Técnica

SEGURANÇA

A IA emitiu chamadas de API com chaves privadas incorporadas diretamente no código do cliente (*front-end*), expondo credenciais de faturação a eventuais ataques. A lógica da Caça ao Tesouro reside inteiramente no cliente (`localStorage`), permitindo que qualquer utilizador forja medalhas e cupões de desconto. Além disso, as dependências npm geradas não foram auditadas, introduzindo riscos graves de vulnerabilidades de *supply chain*.

4. ROADMAP DE ENGENHARIA DE SOFTWARE PARA PRODUÇÃO

Para transformar a demonstração rápida num produto de software profissional, robusto e escalável ao longo do semestre, a equipa executará o seguinte plano de trabalho disciplinado:

FASE 1 • REQUISITOS & MODELAÇÃO

Especificação Formal e UML

Levantamento de stakeholders (visitantes, curadores, loja), especificação de casos de uso, histórias de utilizador com critérios de aceitação em BDD e diagramas de classe/sequência.

FASE 2 • ARQUITETURA & GOVERNANÇA

Design Modular e Registo de ADRs

Desenho de arquitetura desacoplada (REST/GraphQL API + Front-end móvel), isolamento de serviços de IA e documentação formal das decisões tecnológicas (ADRs).

FASE 3 • QUALIDADE & CI/CD

Testes Automatizados e Pipeline

Desenvolvimento orientado a testes (TDD), implementação de testes unitários/E2E com Jest e Cypress, e automatização de integração contínua (CI/CD) com análise estática (SonarQube).

FASE 4 • BACKEND & SEGURANÇA

Autenticação, Validação e Guardas

Migração do estado de gamificação para servidor seguro, autenticação OAuth2/JWT, higienização de inputs no chatbot (anti-prompt injection) e encriptação de tokens QR Code.

Lição Central do Desafio

"A Inteligência Artificial Generativa permite produzir código bruto e protótipos funcionais em escassos minutos. Contudo, a Engenharia de Software é a disciplina indispensável de compreender problemas, projetar arquiteturas evolutivas, garantir a qualidade rigorosa, testar e manter sistemas fiáveis e seguros ao longo do tempo."